



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

SEMESTER 1

2025/2026

SECJ3263-02 DATABASE PROGRAMMING

Project II

Made by:

NAME	MATRIC NUMBER
DAYANG FARAH FARZANA BINTI ABANG IDHAM	A23CS0071
FARRA NURZAHIN BINTI ZAHARIL ANUAR	A23CS0079
NURUL ADRIANA BINTI KAMAL JEFRI	A23CS0258
SAFIYA NURSYAHADAH BINTI MASNOOR	A23CS0176

LECTURER: DR. SHAMINI A/P RAJA KUMARAN

TABLE OF CONTENT

1.0 Introduction.....	3
1.1 Brief Description of the Chosen Domain.....	3
1.2 Project Objectives.....	3
1.3 Team Members & Roles.....	3
2.0 NoSQL Data Model.....	4
2.1 List and Explanation of Collections.....	4
2.2 Example of JSON documents.....	4
2.3 Explanation of Embedding vs Referencing.....	6
2.4 Data Types Used.....	7
3.0 CRUD Implementation Summary.....	9
3.1 Explanation of insert, query, update, delete Operations.....	9
3.2 Screenshots of MongoDB Compass or Shell Execution.....	10
3.3 Projections and query tests.....	16
4.0 Advanced Operations.....	19
4.1 Index Creation Examples.....	19
4.2 Aggregation Pipeline Examples.....	19
4.3 Sorting Demonstrations.....	20
4.4 Explanation of Performance Improvements.....	22
4.5 Any challenges encountered.....	22
5.0 Security and Limitations.....	23
5.1 Basic Access Controls.....	23
5.2 NoSQL Injection Risks.....	23
5.3 Mitigation Strategies.....	24
5.4 System Limitations.....	24
6.0 Teamwork.....	24
7.0 Conclusion.....	25

1.0 Introduction

This section provides an overview of the Delivery Records System, outlining the project objectives, and the specific roles assigned to each team member.

1.1 Brief Description of the Chosen Domain

This Delivery Records System, is a specialized backend database solution designed to manage the logistical operations of a modern courier service. In a high volume commercial environment, managing data for customers, orders, drivers, and delivery statuses requires a flexible architecture that can handle non-relational workloads effectively. By implementing a NoSQL backend using MongoDB, the system utilizes a Document Data Model to store complex, nested data structures such as multiple saved addresses within single documents. This project focuses exclusively on backend development to demonstrate how document-based storage can support activity management and logging systems without the need for a front-end interface.

1.2 Project Objectives

The primary objective of this project is to design and implement a functional NoSQL database that effectively models real-world delivery scenarios. This involves creating at least three to four MongoDB collections that utilize both embedding and referencing techniques to organize data efficiently. Furthermore, the project aims to demonstrate proficiency in CRUD operations, such as inserting records, querying with filters, and updating data using specific MongoDB operators. To ensure high performance and provide business insights, the system also incorporates advanced operations including the creation of single-field and compound indexes, as well as the development of aggregation pipelines to calculate totals and group data by specific categories. Finally, the project includes an analysis of security considerations, such as access control and injection risks, to ensure the database remains robust against common NoSQL vulnerabilities.

1.3 Team Members & Roles

Members	Roles
Dayang Farah Farzana	Manage “Deliveries” collection documents and operations, Projection and Query Test and Conclusion.
Farra Nurzahin	Manage “Drivers” collection documents and operations, explain CRUD implementation and Challenges Encountered.
Nurul Adriana	Manage “Orders” collection documents and operations, explain embedding vs referencing and Performance Improvements.
Safiya Nursyahadah	Introduction, manage “Customers” collection documents and operations and Security and Limitations

2.0 NoSQL Data Model

2.1 List and Explanation of Collections

Customers

The customers collection manages the personal profiles and membership details of users within the delivery system. It stores core information such as names, contact details, and subscription tiers to differentiate between standard and premium members. To support multiple delivery locations, address data is stored as embedded documents within an array (`saved_address`), allowing users to toggle between labels like “Home” or “Office” easily. This structure is designed to facilitate fast lookups by email and enables the business to perform aggregations to track customer growth and premium membership distribution.

Orders

The orders collection serves as the central transaction log for the Delivery Records System. It captures the details of every purchase made by customers. Instead of storing product data in a separate table (as in SQL), this collection utilizes embedding to store `product_details` directly within the order document. This ensures that the snapshot of the item price and quantity at the time of purchase is preserved. However, the collection uses referencing for the `customer_id` to link back to the customers collection, avoiding the duplication of changing user data like phone numbers or addresses.

Drivers

The drivers collection contains information about the delivery drivers responsible for delivering customer orders. It includes vehicle information, current delivery assignments and driver details including name, vehicle type, rating and joined date. Vehicle information is stored in embedded documents (`vehicle_info`) in this collection and each driver’s multiple active deliveries are represented by arrays (`current_deliveries`). Moreover, the design enables effective querying, performance analysis and driver aggregation based on vehicle type and rating.

Deliveries

The deliveries collection is responsible for tracking the execution and status of delivery operations within the Delivery Records System. This collection links customer orders with assigned drivers and records delivery-related information such as delivery fees, status and timestamps. Unlike customers and orders collections, the deliveries collection applies referencing to maintain relationships with other collections. Each delivery document stores references to `order_id` and `driver_id`, allowing the system to associate delivery activities with the correct order and driver without duplicating large datasets. This approach improves data consistency and reduces redundancy, especially when order or driver details change.

2.2 Example of JSON documents

Customers

```
  _id: ObjectId('6961f52cba0447b9dc027c8b')
  customer_id: "C001"
  name: "Kang Taehyun"
  email: "taehyun@gmail.com"
  phone: "011-99998888"
  joined_date: 2024-01-10T00:00:00.000+00:00
  saved_address: Array (3)
    0: Object
      label: "Home"
      address: "No 15, Jalan Sg Buaya, Rawang"
    1: Object
      label: "Office"
      address: "Perodua, Sg Choh"
    2: Object
      label: "Gym"
      address: "Sri Hartamas, KL"
  is_premium: true
```

Orders

```
  _id: ObjectId('6962c9fad586984bc3a5ba71')
  order_id: "ORD1001"
  customer_id: "C001"
  order_date: 2025-12-01T10:00:00.000+00:00
  total_amount: 18.5
  status: "Completed"
  product_details: Array (2)
    0: Object
      product_name: "Corrugated Box (M)"
      qty: 5
      price: 2.5
    1: Object
      product_name: "Bubble Wrap Roll (1m)"
      qty: 1
      price: 6
```

Drivers

```

_id: ObjectId('695e821bbf393021d4e44190')
driver_id: "D001"
name: "Ahmad"
vehicle_type: "Motorcycle"
rating: 4.7
joined_date: "2024-08-10"
▼ vehicle_info: Object
  plate_number: "JQK1234"
  capacity: 15
▼ current_deliveries: Array (2)
  0: "ORD1001"
  1: "ORD1003"

```

Deliveries

```

▼ {
  ▼ "_id": {
    "$oid": "6962283dbb8e0594372cd46d"
  },
  "delivery_id": "DEL001",
  "order_id": "ORD1001",
  "driver_id": "D001",
  "delivery_status": "Completed",
  "delivery_fee": 6.5,
  ▼ "delivery_date": {
    "$date": "2024-08-12T14:30:00.000Z"
  },
  "payment_method": "Cash"
}

```

2.3 Explanation of Embedding vs Referencing

For the orders collection, a hybrid data modeling approach was selected to balance data integrity with performance. We utilized embedding for the `product_details` field (containing item names, quantities, and prices) because these details represent a historical snapshot of the purchase. If a product's price changes in the future, the price recorded in the past order must remain unchanged for accounting purposes; embedding ensures this data remains immutable within the order document. Conversely, we applied referencing for the `customer_id` field. Since customer information such as addresses and phone numbers may change over time, referencing the

customers collection prevents data duplication and ensures that updates to a user's profile are automatically reflected across all their associated orders without needing to update every historical transaction.

2.4 Data Types Used

Customers

Field Name	Data Type	Description
customer_id	String	Unique identifier for each customer.
name	String	Full name of the customer.
email	String	Email of the customer.
phone	String	Phone number of the customer.
joined_date	Date	The date the account was created.
saved_address	Array (Object)	List of delivery locations with labels and addresses.
is_premium	Boolean	Flag indicating if the user has a premium subscription.

Orders

Field Name	Data Type	Description
order_id	String	Unique identifier for each order.
customer_id	String	Reference to customer collection.
order_date	Date	The timestamp when the order was placed.
total_amount	Double	The total cost of the order.
product_details	Array (Object)	A list of items purchased, including name, quantity (qty), and price.
status	String	Current state of the order (e.g., Pending, Shipped).

Drivers

Field Name	Data Type	Description
------------	-----------	-------------

driver_id	String	Unique identifier for each driver.
name	String	Full name of the driver.
vehicle_type	String	Type of vehicle the driver uses.
plate_number	String	Vehicle registration number.
rating	Double	Driver's performance rating on a scale 1-5.
capacity	Integer	Maximum load or seating capacity of the vehicle.
joined_date	Date	The date when the driver joined the company.
current_deliveries	Array	List of current delivery IDs assigned to the driver.
vehicle_info	Object	Detailed information about the vehicle.

Deliveries

Field Name	Data Type	Description
delivery_id	String	Unique identifier for each delivery.
order_id	String	Reference to orders collection.
driver_id	String	Reference to drivers collection.
delivery_status	String	Current status such as Pending, Completed or Cancelled.
delivery_fee	Number	Fee charged for the delivery.
delivery_date	Date	Timestamp of the delivery.
payment_method	String	Method used for payment.

3.0 CRUD Implementation Summary

3.1 Explanation of insert, query, update, delete Operations

Insert Operations

The `insertOne()` method in Figure 3.1 is used to add a single customer document into the collection, which is ideal for individual user registrations. In addition to this, the insert operation is used to update the orders collection with new order records. When multiple orders are entered at once, the `insertMany()` operation in Figure 3.2 is utilized in this project to efficiently insert bulk data.

Query Operations

The customers collection query effectively retrieves data by using the `find()` function in Figure 3.3 with a certain filter and projection. The projection guarantees that only necessary fields such as name and emails are returned. This method reduces data overhead and guarantees that results do not include sensitive information.

Furthermore, the query process is used in situations where the driver data is retrieved. Condition filters like `$gt` in Figure 3.4 and `$in` in Figure 3.5 are used with the `search()` method to retrieve drivers whose ratings are higher than a given value and vehicle type, respectively. These searches are performed to find drivers who use particular vehicle classification and drivers who perform well.

Moreover, the query in Figure 3.6 uses a combination of logical operators `$and` and `$or` to conduct a particular search on the deliveries collection. While the nested `$or` operator offers the ability to match a document if it meets at least one of the requirements between “Completed” or “Pending” status, the `$and` operator offers that multiple conditions must be satisfied between the status and the delivery fee.

Update Operations

The `updateOne()` method in Figure 3.7 combined with the `$set` operator allows the system to modify specific fields without rewriting the entire document. For example, as an order moves through the logistics chain, its status field is updated from “Pending” to “Shipped” or “Completed” in real-time.

Delete Operations

The `deleteMany()` method in Figure 3.8 is used in this operation to eliminate the several documents from a collection according to a particular filter. In this project, all delivery that has the “Cancelled” status was deleted. This filter ensures that the database stays clear of unnecessary data by enabling the system to clean up useless entries in a single operation.

3.2 Screenshots of MongoDB Compass or Shell Execution

Insert Operations

1. insertOne()

```
> db.customers.insertOne({
  "customer_id": "C006",
  "name": "Lee Jihoon",
  "email": "jihoon@gmail.com",
  "phone": "019-5556667",
  "joined_date": new Date(),
  "saved_address": [
    { "label": "Home", "address": "Jalan Ampang, KL" }
  ],
  "is_premium": false
})
< {
  acknowledged: true,
  insertedId: ObjectId('6962186bba0447b9dc027c90')
}
```

Figure 3.1: insertOne() method

2. insertMany()

```
delivery_db> db.orders.insertMany([
  {
    "order_id": "ORD1001",
    "customer_id": "C001",
    "order_date": new Date("2025-12-01T10:00:00Z"),
    "total_amount": 18.50,
    "status": "Completed",
    "product_details": [
      { "product_name": "Corrugated Box (M)", "qty": 5, "price": 2.50 },
      { "product_name": "Bubble Wrap Roll (1m)", "qty": 1, "price": 6.00 }
    ]
  },
  {
    "order_id": "ORD1002",
    "customer_id": "C002",
    "order_date": new Date("2025-12-02T14:30:00Z"),
    "total_amount": 45.00,
    "status": "Pending",
    "product_details": [
      { "product_name": "Premium Poly Mailer (100pcs)", "qty": 1, "price": 45.00 }
    ]
  },
  {
    "order_id": "ORD1003",
    "customer_id": "C001",
    "order_date": new Date("2025-12-05T09:15:00Z"),
    "total_amount": 12.00,
    "status": "Shipped",
    "product_details": [
      { "product_name": "Fragile Stickers (Roll)", "qty": 2, "price": 6.00 }
    ]
  },
  {
    "order_id": "ORD1004",
    "customer_id": "C003",
    "order_date": new Date("2025-12-06T11:20:00Z"),
    "total_amount": 30.00,
    "status": "Pending",
    "product_details": [
      { "product_name": "A4 Document Flyer", "qty": 50, "price": 0.60 }
    ]
  },
])
```

```

{
  "order_id": "ORD1005",
  "customer_id": "C002",
  "order_date": new Date("2025-12-07T08:45:00Z"),
  "total_amount": 8.50,
  "status": "Completed",
  "product_details": [
    { "product_name": "Clear Packing Tape", "qty": 2, "price": 3.00 },
    { "product_name": "Permanent Marker", "qty": 1, "price": 2.50 }
  ]
}

```

Figure 3.2: insertMany() method

Query Operations

1. find()

```

> db.customers.find(
  { "is_premium": true },
  { "name": 1, "email": 1, "_id": 0 }
)
< {
  name: 'Kang Taehyun',
  email: 'taehyun@gmail.com'
}
{
  name: 'Huening Kai',
  email: 'kai@gmail.com'
}
{
  name: 'Choi Yeonjun',
  email: 'yeonjun@gmail.com'
}

```

Figure 3.3: find() method

2. Condition filters: \$gt

```
> db.drivers.find({ rating: { $gt: 4.5 } })
< {
  _id: ObjectId('695e821bbf393021d4e44190'),
  driver_id: 'D001',
  name: 'Ahmad',
  vehicle_type: 'Motorcycle',
  rating: 4.7,
  joined_date: '2024-08-10',
  vehicle_info: {
    plate_number: 'JQK1234',
    capacity: 15
  },
  current_deliveries: [
    'ORD1001',
    'ORD1003'
  ]
}
{
  _id: ObjectId('695f80bcd4513b85d4931c'),
  driver_id: 'D004',
  name: 'Ali',
  vehicle_type: 'Motorcycle',
  rating: 4.9,
  joined_date: 2024-07-20T00:00:00.000Z,
  vehicle_info: {
    plate_number: 'WXY9911',
    capacity: 18
  },
  current_deliveries: [
```

Figure 3.4: \$gt condition filter

3. Condition filters: \$in

```
> db.drivers.find({ vehicle_type: { $in: ["Motorcycle", "Car"] } })
< {
  _id: ObjectId('695f80bcd4513b85d4931b'),
  driver_id: 'D003',
  name: 'Sofi',
  vehicle_type: 'Car',
  rating: 4.5,
  joined_date: 2024-09-05T00:00:00.000Z,
  vehicle_info: {
    plate_number: 'JNP9010',
    capacity: 25
  },
  current_deliveries: [
    'ORD1005',
    'ORD1006',
    'ORD1007'
  ]
}
{
  _id: ObjectId('695f80bcd4513b85d4931e'),
  driver_id: 'D006',
  name: 'Hafiy',
  vehicle_type: 'Car',
  rating: 4.6,
  joined_date: 2024-06-15T00:00:00.000Z,
  vehicle_info: {
    plate_number: 'CAR5566',
    capacity: 30
  },
}
```

Figure 3.5: \$in condition filter

4. Condition filters: \$and, \$or

```
> db.deliveries.find({
  $and: [
    {
      $or: [
        { delivery_status: "Completed" },
        { delivery_status: "Pending" }
      ]
    },
    { delivery_fee: { $gt: 5 } }
  ]
})
< {
  _id: ObjectId('69623105131a4a93f64020cf'),
  delivery_id: 'DEL001',
  order_id: 'ORD1001',
  driver_id: 'D001',
  delivery_status: 'Completed',
  delivery_fee: 6.5,
  delivery_date: 2024-08-12T14:30:00.000Z,
  payment_method: 'Cash'
}
```

Figure 3.6: \$and and \$or condition filter

Update Operations

1. updateOne(), with condition filter using \$set

```
> db.orders.updateOne(
  { "order_id": "ORD1002" },
  {
    $set: { "status": "Shipped" },
    $currentDate: { "last_updated": true }
  }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Figure 3.7: updateOne() update operation

Delete Operations

1. deleteMany()

```
> db.deliveries.deleteMany({
  delivery_status: "Cancelled"
})
< {
  acknowledged: true,
  deletedCount: 1
}
```

Figure 3.8 : deleteMany() delete operation

3.3 Projections and query tests

Projection

Projection is used to control which fields are returned in a query result. In the deliveries collection, projection is applied to display only essential delivery information such as the delivery status and delivery fee, while excluding internal identifiers. This improves query

efficiency and ensures that only relevant data is shown to the user. Figure 3.9 below shows the example of projection and its result.

```
> db.deliveries.find(
  { delivery_status: "Completed" },
  {
    _id: 0,
    delivery_id: 1,
    delivery_status: 1,
    delivery_fee: 1
  }
)
< {
  delivery_id: 'DEL001',
  delivery_status: 'Completed',
  delivery_fee: 6.5
}
```

Figure 3.9 : Example of Projection

Query Test

A query test is conducted using conditional filtering to retrieve delivery records that have a delivery fee greater than a specified amount. This type of query is useful for identifying high-value deliveries and analyzing fee distributions. Figure 3.10 below illustrates the query test using a conditional filter and its result.

```
> db.deliveries.find({
  $and: [
    { delivery_status: "Completed" },
    { delivery_fee: { $gt: 5 } }
  ]
})
< {
  _id: ObjectId('69623105131a4a93f64020cf'),
  delivery_id: 'DEL001',
  order_id: 'ORD1001',
  driver_id: 'D001',
  delivery_status: 'Completed',
  delivery_fee: 6.5,
  delivery_date: 2024-08-12T14:30:00.000Z,
  payment_method: 'Cash'
}
```

Figure 3.10 : Example of Query Test

4.0 Advanced Operations

4.1 Index Creation Examples

As shown in Figure 4.1, a single-field index was created for the email attribute using the `createIndex()` command. This field was selected because it is the primary identifier used during login and user searches.

By indexing the email field, the database avoids time consuming “collection scans” that examine every document. Instead, it uses a sorted structure to locate records instantly. This optimization significantly improves query performance, reduces server load, and ensures the system remains fast as more customers are added to the database.

```
> db.customers.createIndex({ "email": 1 })  
< email_1
```

Figure 4.1: Create index using `createIndex()`

```
> db.orders.createIndex({ "customer_id": 1, "order_date": -1 })  
< customer_id_1_order_date_-1
```

Figure 4.2: Creating an index to search by Customer and sort by Date.

4.2 Aggregation Pipeline Examples

Figure 4.3 shows the aggregate query on the drivers collection that uses a pipeline by filtering and grouping data. The `$match` operation looks for drivers who have a rating higher than 4.0. The `$group` operation then groups these drivers according to their `vehicle_type` and uses the `$sum` attribute to determine the overall count for each category.

```
> db.drivers.aggregate([ { $match: { rating: { $gt: 4.0 } } }, { $group: { _id: "$vehicle_type", total_drivers: { $sum: 1 } } } ] )  
< {  
  _id: 'Van',  
  total_drivers: 1  
}  
{  
  _id: 'Motorcycle',  
  total_drivers: 3  
}  
{  
  _id: 'Car',  
  total_drivers: 2  
}
```

Figure 4.3: Aggregation on the drivers collection

Figure 4.4 shows an aggregation pipeline is developed to calculate the total and average delivery fees collected from completed deliveries. In this implementation, the pipeline filters delivery records with a status of Completed (such as DEL001) and then groups the data to compute the required values. Explanation of aggregation pipeline stages are as follows :

\$match - Filters delivery documents where delivery_status is set to Completed.

\$group - Aggregates the delivery_fee field to calculate the total and average delivery fees collected.

```
> db.deliveries.aggregate([
  {
    $match: { delivery_status: "Completed" }
  },
  {
    $group: {
      _id: null,
      totalDeliveryFee: { $sum: "$delivery_fee" },
      averageDeliveryFee: { $avg: "$delivery_fee" }
    }
  }
])
< {
  _id: null,
  totalDeliveryFee: 6.5,
  averageDeliveryFee: 6.5
}
```

Figure 4.4: Aggregation on deliveries collection

4.3 Sorting Demonstrations

Figure 4.5 displays the driver_id and rating after retrieving every item from the drivers collection. The drivers are then arranged in ascending order according to their rating (ranging from 4.1 to 4.9) using the sort() method with a specification of 'rating:1'. While Figure 4.6 displays the drivers arrangements rating in descending order also using the sort() method but with 'rating:-1'. This makes it simple to find top-rated employees while maintaining a focused and uncluttered output by providing a clear driving performance.

```
> db.drivers.find({}, {_id:0, driver_id:1, rating:1}).sort({rating:1})
< {
  driver_id: 'D005',
  rating: 4.1
}
{
  driver_id: 'D002',
  rating: 4.3
}
{
  driver_id: 'D003',
  rating: 4.5
}
{
  driver_id: 'D006',
  rating: 4.6
}
{
  driver_id: 'D001',
  rating: 4.7
}
{
  driver_id: 'D004',
  rating: 4.9
}
}
```

Figure 4.5: Ascending order sorting

```
> db.drivers.find({}, {_id:0, driver_id:1, rating:1}).sort({rating:-1})
< {
  driver_id: 'D004',
  rating: 4.9
}
{
  driver_id: 'D001',
  rating: 4.7
}
{
  driver_id: 'D006',
  rating: 4.6
}
{
  driver_id: 'D003',
  rating: 4.5
}
{
  driver_id: 'D002',
  rating: 4.3
}
{
  driver_id: 'D005',
  rating: 4.1
}
```

Figure 4.6: Descending order sorting

4.4 Explanation of Performance Improvements

To optimize query performance, a Compound Index was created on { “customer_id: 1, “order_date”: -1 }. This indexing strategy was chosen because a primary use case for the application is displaying a user’s recent activity, where the system must find all orders belonging to a specific customer and display them starting from the newest. Without this index, the Delivery Records System would be forced to perform a full collection scan to find the customer’s documents and then perform an expensive in-memory sort operation to arrange them by date. The compound index allows the database to locate the specific customer’s records and read them in the correct pre-sorted order instantly, significantly reducing query execution time and server load.

4.5 Any challenges encountered

Although MongoDB provided a great deal of flexibility in handling delivery data, it also brought particular challenges with regard to query logic relationship management. The main implementation related obstacles and solutions are described in depth in this section.

4.5.1 Maintaining Data Accuracy

One of the primary issues was ensuring data consistency across referenced collections. As MongoDB does not have strict foreign key constraints, manual coordination of identifiers (such as `order_id`, `customer_id` and `driver_id`) was necessary to establish adequate linkages between the order, drivers and deliveries collections

4.5.2 Constructing Aggregation Pipelines

The multi-stage processing architecture of MongoDB made it challenging to create and debug aggregation pipelines. As each stage including `$match`, `$group` and `$project` transfers its modified output to the next, a single logic error early in the process could lead to incorrect final results or empty datasets.

5.0 Security and Limitations

This section evaluates the security framework of the NoSQL environment, focusing on the protection of delivery data and the inherent constraints of the non-relational model.

5.1 Basic Access Controls

The security framework for the Delivery Records System is built on Role-Based Access Control (RBAC) to ensure that data access is restricted to authorized personnel only. In a realistic deployment, different team members have different access levels. For example, while the Drivers collection managed by Farra Nurzahin allows staff to view their specific delivery tasks, they do not have permission to view sensitive customer data. As the Customer Specialist, it has the administrative privileges to execute `insertOne()` and `updateOne()` on customer profiles. This hierarchical approach prevents unauthorized users from modifying membership statuses or private contact details via the mongosh shell, ensuring that the customer's identity data remains protected from internal misuse.

5.2 NoSQL Injection Risks

A primary security threat to this system is NoSQL Injection, where malicious code is embedded into input fields to manipulate database queries. In the context of the Customers collection, if the `find()` operation used to search for emails is not properly secured, an attacker could input logical operators like `{"$ne": null}`. This would force the database to return all records that are "not null," effectively bypassing authentication and exposing the entire user list. Such an attack could lead to a massive data breach, compromising the privacy of every individual in the system.

5.3 Mitigation Strategies

To defend against these risks, we implement several layers of protection. First, Input Validation is applied to ensure all data, such as phone numbers or the order details handled by Nurul Adriana, follows a strict format before being saved. Second, we use Query Sanitization, which treats user input as literal text rather than executable commands, preventing attackers from "injecting" operators into the search logic. Finally, the Principle of Least Privilege ensures that even if one part of the system is compromised, the damage is limited because each role (such as Driver or Order Manager) only has access to the specific collection they need to perform their job.

5.4 System Limitations

Despite its efficiency, the document-oriented model has inherent limitations. The Embedding strategy used for the `saved_address` array is subject to MongoDB's 16MB document size limit. If a customer saves an excessive number of locations, the document could eventually fail to save. Additionally, because the system relies on manual Referencing to link the Customers, Drivers, and Orders collections, the work done by Dayang and Adriana, it lacks the automatic "Cascading Delete" functionality of relational databases. This means if a customer is deleted, the system does not automatically remove their old orders or delivery logs, requiring manual maintenance to

prevent "orphaned" data from cluttering the database.

6.0 Teamwork

Dayang Farah Farzana

I was responsible for managing the Deliveries collection in the Delivery Records System. My contributions included designing the deliveries data model using referencing techniques to link orders and drivers, as well as preparing sample JSON documents to ensure data consistency across collections. I implemented CRUD operations such as deleting cancelled delivery records and performing complex queries using logical operators. In addition, I developed an aggregation pipeline to calculate total and average delivery fees, demonstrating the use of advanced MongoDB operations. I also contributed to projection and query testing, and completed the project conclusion by summarizing key learnings, challenges, and recommendations for improvement.

Farra Nurzahin

I was in charge of designing and implementing the drivers collection for this project which monitors staff performance and vehicle assignments for the Delivery Record System. My main contribution was modelling the driver data structure and applying filtering logic to separate high-performance employees according to the particular vehicle classifications and ratings utilizing condition filters like `$gt` and `$in`. Furthermore, I performed operations particularly managing aggregation pipelines that calculate driver totals using `$match` and `$group` and sorting drivers based on the rating in ascending and descending order. I also contributed to the documentation by listing and explaining all the collections and described the technical challenges encountered during development.

Nurul Adriana

In this project, I was responsible for the design and implementation for the design and implementation of the orders collection, which serves as the central transaction log for the Delivery Records System. My primary contribution involved modeling the data structure to effectively utilize the system's embedding capabilities for product details while maintaining relational integrity through customer references. I executed the core CRUD operation, specifically handling bulk data insertion using `insertMany()` and status updates via `updateOne()`. Additionally, I implemented a compound index on customer IDs and order dates to optimize query performance for user history lookups. I also contributed to the documentation by detailing the rationale behind our schema design choices and the performance benefits of our indexing strategy.

Safiya Nursyahadah

I was responsible for the design and implementation of the customers collection, which serves as the primary repository for user profiles and membership data within the Delivery Records System. My main contribution involved modelling the data structure to effectively utilize MongoDB's embedding capabilities for saved addresses, ensuring that multiple delivery locations are stored efficiently within a single document. I executed core CRUD operations, specifically focusing on individual data entry using `insertOne()` and retrieving filtered datasets through the use of projection to optimize data visibility. Additionally, I implemented a single-field index on the email attribute to enhance query performance during user

authentication. I also contributed to the documentation detailing the security measures and the introduction of this Delivery Records System.

7.0 Conclusion

In conclusion, this project successfully demonstrated the design and implementation of a NoSQL-based Delivery Records System using MongoDB. Through the development of multiple collections such as customers, orders, drivers and deliveries, the project provided practical experience in applying document-oriented data modeling concepts, including embedding and referencing. The implementation of CRUD operations, projections, indexing, sorting and aggregation pipelines enhanced understanding of how MongoDB handles real-world data scenarios efficiently.

Throughout this project, several challenges were encountered, particularly in maintaining data consistency across referenced collection and constructing accurate aggregation pipelines. Since MongoDB does not force foreign key constraints, careful coordination of identifiers was required to ensure correct relationships between orders, drivers and deliveries.

For future improvements, the system could be enhanced by implementing \$lookup operations to generate more comprehensive reports that combine data from multiple collections. The use of schema validation rules could also improve data consistency and reduce the risk of malformed documents. Overall, this project provided valuable hands-on experience in NoSQL database design and highlighted the flexibility and scalability of MongoDB for managing delivery-based systems.

Video URL -

<https://drive.google.com/file/d/1tG9RqNCqIdbgjZoe8ue7ibtfmJCyyApR/view?usp=sharing>